

Zartags Writings

Ausarbeitung zum Web-Entwicklungsprojekt

Ronny Wittmer (Matrikelnummer: 313387)

Melissa Armbruster (Matrikelnummer: 313275)

Abgabedatum: 25. Juli 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Projektidee	4
1.2	Motivation	4
1.3	Zielgruppe	4
2	Technologie-Stack	5
2.1	Frontend	5
2.2	Backend	5
2.3	Datenhaltung	6
2.4	Entwicklungs- und Testwerkzeuge	7
3	Architektur	8
3.1	Komponentenstruktur	8
3.2	API-Design	9
3.3	Datenbankschema	10
3.4	Diagramme	11
4	Umsetzung pro Meilenstein	14
4.1	Meilenstein 1	14
4.2	Meilenstein 2	15
4.3	Meilenstein 3	16
4.4	Meilenstein 4	18
4.5	Zuordnung von VL-Konzepten zum Code	19
5	Testing & Qualitätssicherung	21
5.1	Teststrategie	21
5.2	Was wird getestet	21
5.3	Exemplarische Ergebnisse	22
6	Betrieb	23
6.1	Start der Anwendung	23
6.2	Konfiguration	23
6.3	Betriebsmodus	24
7	Reflexion & Fazit	25
7.1	Was lief gut	25
7.2	Was würden wir anders machen	25
7.3	Was wurde gelernt	26
A	Anhang: Screenshots der fertigen App	27
B	Formales und Quellenhinweise	28

B.1	Formale Anforderungen	28
B.2	Quellen	28

Abbildungsverzeichnis

1	Komponentendiagramm der Anwendung	11
2	Sequenzdiagramm: Login und geschützter Datenabruf	11
3	Vereinfachtes ER-Diagramm des Domänenmodells	12
4	Autorisierungs- und Request-Flow für schreibende Endpunkte	12
5	Platzhalter: Übersichtsseite der Anwendung	27

Tabellenverzeichnis

1	Zentrale API-Endpunkte	9
2	Zuordnung zentraler Vorlesungskonzepte zu Codebereichen	20
3	Relevante Umgebungsvariablen im Entwicklungsbetrieb	24

Codeverzeichnis

1	Backend-Testlauf	22
2	Frontend-Testlauf	22
3	Ausgewählte gezielte Testläufe	22
4	Backend lokal starten	23
5	Frontend lokal starten (zweites Terminal)	23

1 Einleitung

1.1 Projektidee

Zartags Writings ist ein Tool zur strukturierten Verwaltung von Kampagneninhalten für Pen-and-Paper-Rollenspiele. Die Anwendung wurde mit dem Ziel geplant, während einer laufenden Spielrunde schnell Notizen erfassen und übersichtlich organisieren zu können. Im Mittelpunkt steht eine einfache Bedienung, da die Nutzung typischerweise parallel zum Spielgeschehen erfolgt und nicht den Spielfluss unterbrechen soll.

1.2 Motivation

Das Projekt entstand aus eigener Praxis: Melissa übernimmt in der Gruppe die Rolle der Dungeon Masterin, Ronny ist Spieler. Für die gemeinsame Kampagne wurde eine Lösung gesucht, mit der Notizen strukturiert, schnell auffindbar und für alle Beteiligten zentral verfügbar sind. Der Name „Zartag“ stammt dabei aus einem Charakter der eigenen Kampagne.

Ein zentrales Problem im Spielalltag ist, dass Informationen oft verteilt in Chats, Dokumenten oder einzelnen Notizbüchern liegen. Dadurch gehen Zusammenhänge verloren, wichtige Details sind schwer wiederzufinden und der Überblick zwischen Sessions leidet. *Zartags Writings* adressiert dieses Problem durch eine einheitliche Kampagnenstruktur mit gemeinsamem Zugriff sowie der Möglichkeit, Inhalte sichtbar bzw. unsichtbar zu schalten, sodass der DM Informationen vorbereiten kann, bevor sie für alle freigegeben werden.

1.3 Zielgruppe

Die Anwendung richtet sich an alle Personen, die eine Pen-and-Paper-Kampagne leiten oder daran teilnehmen. Primäre Zielgruppen sind Dungeon Master und Spielerinnen bzw. Spieler.

Das Nutzungskonzept ist bewusst flexibel: Auch wenn nicht jede Person selbst Notizen schreiben möchte, kann eine Person aus der Gruppe die Dokumentation übernehmen und die Inhalte für alle anderen zentral bereitstellen. Damit eignet sich die Anwendung sowohl für aktiv mitschreibende Gruppen als auch für Gruppen, in denen nur ein Teil der Mitglieder Inhalte pflegt.

2 Technologie-Stack

Der Technologie-Stack deckt die Anforderungen des Projekts (*Single-Page-Webanwendung mit Login, Rollen/Rechten und persistenter Datenhaltung*) mit überschaubarem Betriebsaufwand abdeckt. Im Fokus stehen dabei: schnelle Entwicklung, gute Testbarkeit und eine klare Trennung zwischen Benutzeroberfläche, API-Logik und Persistenz.

2.1 Frontend

Das Frontend basiert auf **React** mit **TypeScript** und wird über **Vite** gebaut und lokal ausgeführt.

React React passt hier, weil die Anwendung stark zustandsgetrieben ist: Anmeldestatus, Rollen im Campaign-Kontext, Entity-Listen und Bearbeitungszustände müssen konsistent über mehrere Seiten hinweg verwaltet werden. Durch die komponentenbasierte Struktur konnten wiederverwendbare UI-Bausteine (*Layout, Header, Campaign-Komponenten*) klar getrennt und wartbar umgesetzt werden.

TypeScript im Frontend TypeScript reduziert Integrationsfehler zwischen API und UI, insbesondere bei verschachtelten Nutzerdaten und Campaign-Strukturen. Das Projekt nutzt explizite Typen für zentrale Datenobjekte (z. B. *AuthUser, Campaign-API-Typen*), was die Weiterentwicklung erleichtert und Refactorings sicherer macht. Gerade bei rollenabhängiger Darstellung (*DM/EDITOR/PLAYER*) verbessert das die Verständlichkeit der Logik deutlich.

React Router Für die Navigation wird **react-router-dom** eingesetzt. Als Single-Page-Ansatz vermeidet dies vollständige Seiten-Reloads und erlaubt eine saubere Trennung zwischen öffentlichen Seiten (*Login/Registrierung*) und geschützten Bereichen (*Campaign-Ansichten, Verwaltung*). Dadurch bleibt die Nutzerführung schnell und konsistent.

Vite als Build- und Dev-Server Vite passt als Build-Tool, da es in der Entwicklung sehr kurze Start- und Aktualisierungszeiten liefert. Für dieses Projekt ist zusätzlich der konfigurierte `/api`-Proxy entscheidend: Frontend-Anfragen werden im Dev-Betrieb transparent an das Backend weitergeleitet. Das vereinfacht die lokale Entwicklung und vermeidet CORS-Probleme durch abweichende Ports.

2.2 Backend

Das Backend wurde mit **Node.js**, **Express** und **TypeScript** umgesetzt. Es stellt die REST-API bereit, kapselt die Geschäftslogik (*Authentifizierung, Rollenprüfung, Campaign-Operationen*) und steuert den Zugriff auf die Datenbank.

Node.js Node.js passt gut zum Frontend-Stack, da Sprache und Ökosystem über beide Schichten hinweg konsistent sind. Das reduziert Reibungsverluste im Team, vereinfacht Tooling und ermöglicht eine schnelle Iteration in den Meilensteinen.

Express Express ist als bewusst schlankes Framework hier passend. Für die Projektanforderungen — klar definierte REST-Endpunkte, Cookie-basierte Authentifizierung, Middleware für CORS/JSON/Parser — liefert Express genug Struktur, ohne unnötige Komplexität einzuführen. Die Routen sind nach Domänen getrennt (`authRoutes`, `userRoutes`, `campaignRoutes`, `entityRoutes`, `memberRoutes`), was die Wartbarkeit erhöht.

Auth- und Security-Bausteine Für Authentifizierung und Sicherheit werden mehrere spezialisierte Bibliotheken kombiniert:

- **jsonwebtoken:** Signieren und Verifizieren von JWTs,
- **cookie-parser:** Zugriff auf den Token im `HttpOnly-Cookie`,
- **bcrypt:** sicheres Hashing von Passwörtern,
- **cors:** kontrollierte Freigabe erlaubter Origins.

Die Entscheidung für JWT im `HttpOnly-Cookie` reduziert das Risiko, Tokens im Frontend-Code oder Local Storage offenzulegen.

2.3 Datenhaltung

Die Persistenz basiert auf **SQLite** in Kombination mit **Prisma** als ORM.

SQLite SQLite passt, weil die Anwendung im Projektkontext lokal, kompakt und ohne externen Datenbankserver betrieben werden soll. Für den vorgesehenen Umfang (Einzelinstantz, moderate Datenmengen, Entwicklungs-/Demobetrieb) bietet SQLite ein gutes Verhältnis aus Einfachheit und Zuverlässigkeit. Die Einstiegshürde ist gering, da Setup und Deployment ohne separate DB-Infra auskommen.

Prisma Prisma übernimmt die typsichere Datenzugriffsschicht und bildet das Domänenmodell explizit ab (`User`, `Campaign`, `CampaignMember`, `Entity`, `ContentBlock`, ...). Die wichtigsten Vorteile in diesem Projekt:

- **Klare Modellierung:** Relationen und Constraints sind direkt im Schema dokumentiert.
- **Typsichere Queries:** weniger Laufzeitfehler beim Zugriff aus dem TypeScript-Backend.
- **Migrationsunterstützung:** Schema-Änderungen bleiben versioniert und reproduzierbar.

Migrationskonzept Die Datenbankstruktur wird über Prisma-Migrationen verwaltet (`prisma/migrations/...`). Beim Start in Docker wird die Migration automatisiert ausgeführt (`prisma migrate deploy`), wodurch Entwicklungs- und Laufzeitumgebung konsistent bleiben. Dieses Vorgehen minimiert manuelle Setup-Fehler und erhöht die Reproduzierbarkeit.

2.4 Entwicklungs- und Testwerkzeuge

Neben den Kerntechnologien ergänzen Tools die Codequalität, Entwicklungsfluss und Nachvollziehbarkeit unterstützen.

Tests mit Vitest Sowohl Frontend als auch Backend nutzen **Vitest**. Im Backend werden API-Routen in einer Node-Umgebung getestet, im Frontend Komponenten- und Interaktionslogik in einer `jsdom`-Umgebung. Ergänzend kommen **Testing Library** und **Supertest** zum Einsatz, um UI-Verhalten bzw. HTTP-Endpunkte realitätsnah zu prüfen. Die Entscheidung für denselben Test-Runner in beiden Teilprojekten reduziert Komplexität im Tooling.

Codequalität: ESLint + Prettier Im Frontend sorgen **ESLint** (inkl. React- und TypeScript-Regeln) und **Prettier** für konsistente Codequalität und einheitliche Formatierung. Über **simple-git-hooks** wird vor Commits automatisch linting/formatting ausgeführt. So werden Stil- und Basisqualitätsprobleme früh abgefangen.

TypeScript-Tooling und Dev-Workflow Das Backend nutzt **tsx watch** für schnellen TypeScript-Dev-Betrieb ohne separaten Build-Schritt. Im Frontend übernimmt Vite denselben Zweck. Durch diesen direkten Feedback-Zyklus konnten Features pro Meilenstein inkrementell umgesetzt und getestet werden.

Containerisierung mit Docker Compose Für einen reproduzierbaren Start des Gesamtsystems wird **Docker Compose** genutzt. Frontend und Backend laufen als getrennte Services mit klaren Umgebungsvariablen (`DATABASE_URL`, `JWT_SECRET`, `CORS_ORIGIN`, `VITE_API_PROXY_TARGET`). Dadurch ist das Projekt unabhängig vom lokalen Host-Setup schnell lauffähig und für Abgabe/Demo stabil reproduzierbar.

3 Architektur

3.1 Komponentenstruktur

Die Anwendung folgt einer klassischen **3-Schichten-Struktur: Frontend (Präsentation), Backend (API + Fachlogik) und Persistenz (SQLite über Prisma)**. Die Schichten sind klar getrennt und kommunizieren ausschließlich über HTTP-Requests auf `/api/*`.

Frontend-Schicht Das Frontend ist als Single-Page-Anwendung mit React umgesetzt. Die Routen in `frontend/src/App.tsx` bilden die fachlichen Bereiche ab: Authentifizierung (`/login`, `/register`), Campaign-Übersicht (`/campaigns/:slug`) sowie Entity-Ansichten und Bearbeitung.

Eine zentrale Rolle übernimmt der `JWTAuthContext`: Er hält den globalen Authentifizierungszustand (`user`, `isLoggedIn`, `loading`, `error`), führt Login/Logout aus und lädt den aktuellen Benutzer über `/api/user`. Dadurch bleibt Auth-Logik aus den Seitenkomponenten ausgelagert und wiederverwendbar.

Für API-Aufrufe wird ein gemeinsamer Fetch-Wrapper (`frontend/src/lib/api.ts`) genutzt. Dieser setzt standardmäßig `credentials: include`, damit das JWT-Cookie bei geschützten Requests mitgesendet wird.

Backend-Schicht Das Backend ist modular in Routenpakete aufgeteilt, die in `backend/src/server/app.ts` registriert werden: `authRoutes`, `userRoutes`, `campaignRoutes`, `entityRoutes`, `memberRoutes`, `healthRoutes`.

Die Route-Module kapseln jeweils eine fachliche Domäne:

- **Auth:** Registrierung, Login, Logout,
- **User:** aktuelles Profil lesen/löschen,
- **Campaigns:** anlegen, beitreten, laden, Join-Code regenerieren,
- **Entities:** Templates, Listen, Details und CRUD je Entity-Typ,
- **Members:** Rollen- und Anzeigenamenverwaltung innerhalb einer Campaign.

In `backend/src/server/entities.ts` liegt zentrale Domain-Logik, die von mehreren Routen wiederverwendet wird, z. B.: `loadCampaignForUser`, `serializeEntity`, `canEditCampaign`, `canManageCampaign`. So wird verhindert, dass Autorisierungs- und Transformationslogik in mehreren Dateien dupliziert wird.

Persistenz-Schicht Der Datenzugriff erfolgt ausschließlich über Prisma. Die Prisma-Client-Instanz wird zentral in `config.ts` erzeugt und über den Better-SQLite3-Adapter mit

DATABASE_URL verbunden. Das reduziert Kopplung, da Routen keine eigene DB-Verbindung aufbauen.

3.2 API-Design

Die API ist als **REST-orientierte JSON-API** unter /api umgesetzt. Für geschützte Endpunkte wird die Middleware `authenticateToken` verwendet, die den JWT aus dem `HttpOnly-Cookie` validiert und den User-Kontext an den Request hängt.

Konventionen

- Ressourcenorientierte Pfade, z. B. `/api/campaigns/:slug`.
- HTTP-Methoden gemäß Operation (GET, POST, PUT, PATCH, DELETE).
- Fehlerantworten mit passendem Statuscode und JSON-Fehlermeldung (`{ error: ... }`).
- Rollenprüfung serverseitig vor schreibenden Aktionen.

Methode	Pfad	Schutz	Zweck
POST	<code>/api/register</code>	nein	Benutzer registrieren
POST	<code>/api/login</code>	nein	JWT-Cookie setzen
POST	<code>/api/logout</code>	nein	JWT-Cookie löschen
GET	<code>/api/user</code>	ja	aktuellen Benutzer laden
GET	<code>/api/campaigns</code>	ja	Campaign-Liste des Users
POST	<code>/api/campaigns</code>	ja	neue Campaign erstellen
POST	<code>/api/campaigns/join</code>	ja	per Join-Code beitreten
GET	<code>/api/campaigns/:slug</code>	ja	Campaign inkl. Entities laden
POST	<code>/api/campaigns/:slug/regenerate-join-code</code>	ja (DM/Owner)	Join-Code erneuern
GET/POST/PUT/DELETE	<code>/api/campaigns/:slug/entities/...</code>	ja (rollenabhängig)	Entity-CRUD
PATCH	<code>/api/campaigns/:slug/members/:memberUserId</code>	ja (DM/Owner)	Mitgliedsdaten/Rolle ändern

Tabelle 1: Zentrale API-Endpunkte

Zentrale Endpunkte

Autorisierungslogik Die Autorisierung ist zweistufig aufgebaut:

1. **Authentifizierung** über JWT-Cookie (`authenticateToken`).
2. **Fachliche Berechtigungen** über Rollenprüfungen: `canEditCampaign` erlaubt Owner, DM und Editor; `canManageCampaign` erlaubt Owner und DM.

Damit werden unzulässige Änderungen serverseitig abgefangen, auch wenn ein Client manipulierte Requests sendet.

3.3 Datenbankschema

Das Datenmodell ist auf zwei Kernanforderungen ausgelegt: **(1) Mitglieder- und Rollenverwaltung pro Campaign** und **(2) flexible Inhaltsstruktur für unterschiedliche Entity-Typen**.

Kernentitäten

- **User**: registrierte Benutzer mit Login-Daten.
- **Campaign**: Kampagne mit Owner, Metadaten und Join-Code.
- **CampaignMember**: Zuordnung User ↔ Campaign inkl. Rolle.
- **Entity**: fachliches Objekt (PC, NPC, MAGIC_ITEM, LOCATION).
- **EntityField**: strukturierte Header-Felder einer Entity.
- **ContentBlock**: Inhaltskarten pro Entity (Text, Liste, Attribute, Bild).
- **ContentListItem** und **ContentAttribute**: Unterelemente für Listen und Attribute.

Relationen und Constraints

- Ein User kann in vielen Campaigns Mitglied sein; eine Campaign hat viele Mitglieder (`CampaignMember` als Join-Tabelle).
- Eine Campaign hat viele Entities; eine Entity gehört genau zu einer Campaign.
- Eindeutigkeit wird über fachliche Schlüssel gesichert, u. a. `Campaign.slug`, `Campaign.joinCode`, `User.email` sowie (`campaignId`, `type`, `slug`) für Entities.
- Cascading-Deletes entfernen abhängige Daten konsistent (z. B. Entity → Fields/Blocks).

Das Schema kombiniert damit klare Normalisierung für Beziehungen (User, Campaign, CampaignMember) mit einer flexiblen Content-Struktur für heterogene Kampagneninhalte.

3.4 Diagramme

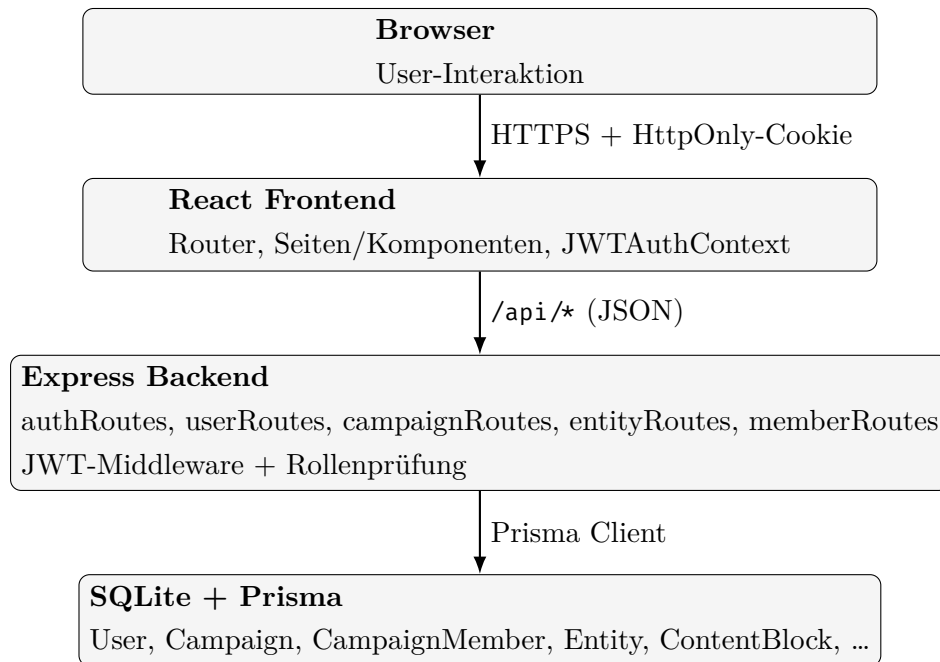


Abbildung 1: Komponentendiagramm der Anwendung

Das Komponentendiagramm zeigt die klare Entkopplung der Schichten. Das Frontend kennt die Datenbank nicht direkt, sondern ausschließlich die HTTP-Schnittstelle. Geschäftsregeln und Rechteprüfung bleiben vollständig im Backend.

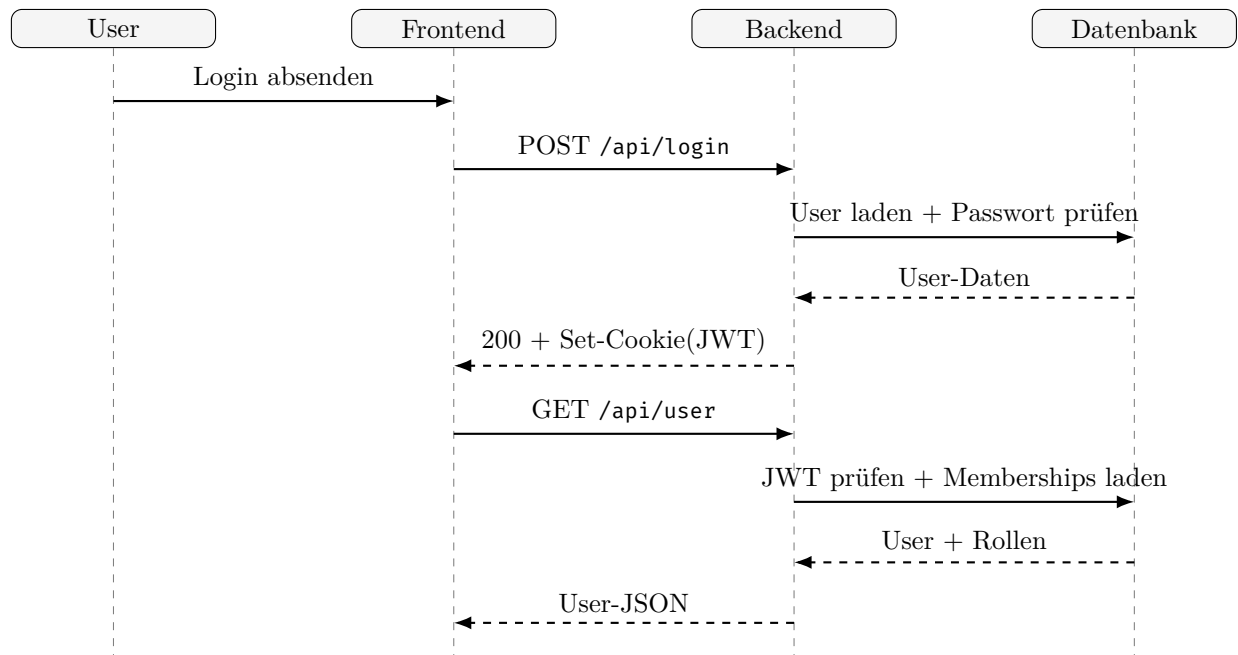


Abbildung 2: Sequenzdiagramm: Login und geschützter Datenabruf

Der Ablauf verdeutlicht, dass der Token nicht im JavaScript-Speicher abgelegt wird, sondern als HttpOnly-Cookie übertragen wird. Folgezugriffe auf geschützte Ressourcen nutzen denselben Mechanismus und werden zusätzlich durch rollenbasierte Prüfungen abgesichert.

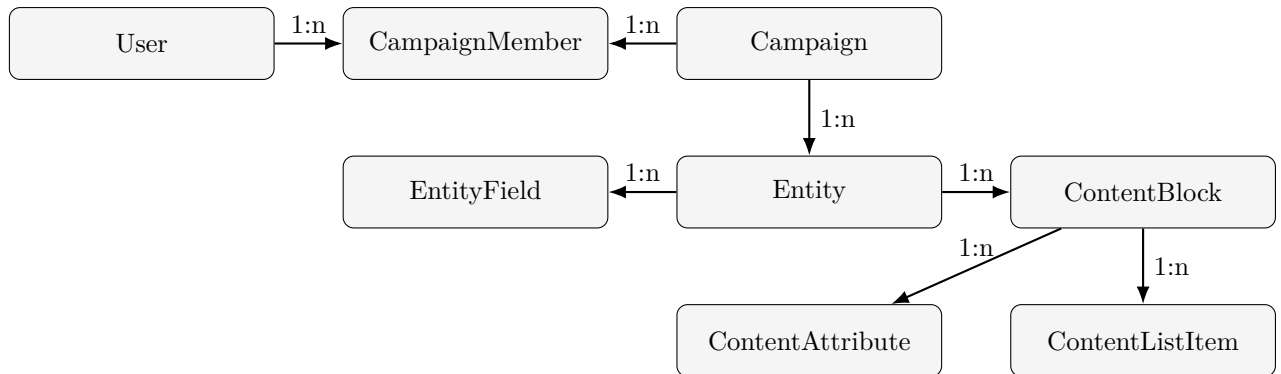


Abbildung 3: Vereinfachtes ER-Diagramm des Domänenmodells

Das ER-Diagramm zeigt die zwei zentralen Stränge der Persistenz: Mitgliedschaft mit Rollen (User–CampaignMember–Campaign) und inhaltliche Modellierung der Campaign-Daten (Campaign–Entity–ContentBlock).

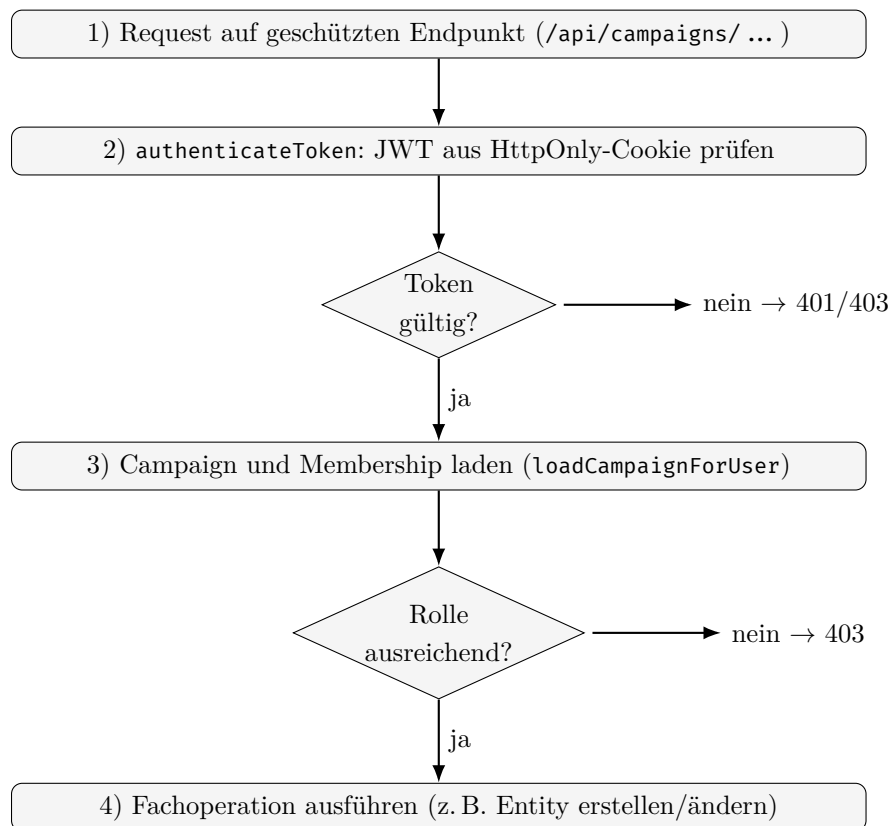


Abbildung 4: Autorisierungs- und Request-Flow für schreibende Endpunkte

Der Ablauf macht sichtbar, dass Autorisierung strikt serverseitig erfolgt: ohne gültigen Token oder ausreichende Rolle wird die Aktion vor der Fachoperation abgelehnt.

4 Umsetzung pro Meilenstein

Die Umsetzung erfolgte inkrementell über vier Meilensteine. Als Referenzstände dienen die getaggtten Commits `meilenstein1`, `Meilenstein2` und `Meilenstein3`; der finale Projektstand entspricht `Meilenstein 4`. Zwischen den Meilensteinen wurde die Anwendung schrittweise von einer statischen Frontend-Struktur zu einer komponentenbasierten SPA und danach zu einer vollständigen Webanwendung mit eigenem Backend, Authentifizierung und persistenter Datenhaltung ausgebaut und schließlich betriebsfertig dokumentiert.

4.1 Meilenstein 1

Umgesetzter Funktionsumfang Im ersten Meilenstein lag der Schwerpunkt auf einem funktionalen Frontend-Grundgerüst in Form statischer Seiten und Templates. Die Kernseiten (`index.html`, `about.html`, `login.html` sowie campaign-spezifische Unterseiten) wurden mit semantischen HTML-Elementen und einer responsiven Navigation umgesetzt.

Typische Bausteine dieses Stands sind:

- semantische Seitenstruktur mit `main`, `section`, `nav` und korrekt beschrifteten Formularfeldern,
- responsive Sidebar-Navigation mit Overlay und Mobile-Breakpoint,
- erste URL-Struktur für Campaign-Bereiche (`/src/campaign1/ ...`),
- grundlegende Inhaltsdarstellung für PCs, NPCs, Locations und Items.

Abgedeckte Anforderungen Die Anforderungen aus der M1-Kriterienmatrix wurden vollständig adressiert: Semantik, Formularstruktur, responsives Layout, Media Queries und konsistente URL-Struktur. Damit entstand eine belastbare UI-Basis, auf der die späteren React-Komponenten fachlich aufbauen konnten.

Verwendete Vorlesungskonzepte In M1 wurden vor allem grundlegende Frontend-Konzepte angewendet:

- **Semantisches HTML:** klare Trennung von Navigation, Inhalt und Formularen.
- **Responsive Design:** Breakpoint-gesteuertes Verhalten für mobile Navigation.
- **Progressive Interaktivität:** Navigation und Sidebar-Verhalten über ein kleines, zielgerichtetes JavaScript-Modul.
- **Accessibility-Basis:** ARIA-Attribute und steuerbare Navigationszustände (`aria-expanded`, `inert`).

Technische Erkenntnisse aus M1 Die statische Struktur war für den Einstieg sinnvoll, zeigte aber früh Grenzen: wiederkehrende Inhaltsbausteine (z. B. Kartenstrukturen) mussten mehrfach gepflegt werden, und Zustände wie Rollen oder Sichtbarkeit konnten nur begrenzt zentral verwaltet werden. Diese Erkenntnisse führten direkt zur Entscheidung, in M2 auf React-Komponenten und zustandsbasierte Datenmodelle umzustellen.

Arbeitsweise und Qualität in M1 Bereits in diesem frühen Stand wurde auf eine klare visuelle und strukturelle Konsistenz geachtet: Navigation, Seitenaufbau und Formulare verhalten sich über die zentralen Seiten einheitlich. Dadurch konnte in den folgenden Meilensteinen die technische Architektur umgebaut werden, ohne den fachlichen Kern der Benutzerführung neu entwerfen zu müssen.

4.2 Meilenstein 2

Fortschritt gegenüber M1 M2 markiert den Architekturwechsel von statischen HTML-Seiten hin zu einer React- und TypeScript-basierten Single-Page-Anwendung. Die zuvor separaten Seiten wurden in wiederverwendbare Komponenten überführt und über `react-router-dom` verbunden.

Zentrale Fortschritte:

- Migration auf **React + TypeScript + Vite**,
- Einführung einer **komponentenbasierten UI-Struktur**,
- Routing für Overview-, Typ- und Edit-Seiten,
- strukturierte Beispieldaten als JSON-basiertes Inhaltsmodell,
- erste rollenabhängige Zustände über einen zentralen Context.

Wichtige technische Entscheidungen 1) **Komponentenzerlegung statt Seitenkopien.** Die Domäne wurde in klar fokussierte Bausteine aufgeteilt, z. B. `CampaignDetail`, `ItemCard`, `ItemsGrid`, `EditableCardContent`. Damit konnte dieselbe Darstellungslogik für unterschiedliche Entity-Typen wiederverwendet werden.

2) **Datengetriebene Darstellung.** Inhalte wurden nicht mehr nur als statisches Markup gepflegt, sondern über ein strukturiertes Datenmodell (Header-Felder, Karten, Listen, Attribute) aufgebaut. Diese Entscheidung reduzierte Redundanz und machte die spätere Backend-Integration vorbereitbar.

3) **Zustandsverwaltung über Context + Hooks.** Mit `AuthContext` wurden editorische Rollen-/Berechtigungszustände zentral modelliert. Gleichzeitig kamen `useState` und `useEffect` systematisch zum Einsatz, um Benutzerinteraktion und persistente UI-Zustände (`localStorage`) konsistent abzubilden.

Verwendete Vorlesungskonzepte M2 adressiert vor allem moderne Frontend-Engineering-Konzepte:

- **SPA-Routing:** dynamische Routen und clientseitige Navigation.
- **Komponentenarchitektur:** Trennung in Presentational- und Feature-Komponenten.
- **State-Management mit Hooks:** lokaler und geteilter Zustand.
- **Typisierung mit TypeScript:** Interfaces/Typen für Daten- und Komponentenverträge.
- **Build-/Tooling-Kette:** Vite als schneller Dev-/Build-Prozess.

Konkrete Codebereiche (Stand **Meilenstein2**)

- Routing und Seitenzuordnung: `src/App.tsx`
- Rollen-/Rechtezustände im Frontend: `src/context/AuthContext.tsx`
- Campaign-Übersicht inkl. Suche und Nutzeraktionen: `src/pages/CampaignOverview.tsx`
- Wiederverwendbare Campaign-Komponenten: `src/components/campaign/*`

Lessons Learned aus M2 Die komponentenbasierte Struktur erhöhte Wartbarkeit und Entwicklungsgeschwindigkeit deutlich, machte aber gleichzeitig deutlich, dass echte Authentifizierung und persistente Mehrbenutzerlogik nicht dauerhaft rein clientseitig modelliert werden sollten. Daraus entstand der nächste konsequente Schritt in M3: eigenes Backend mit serverseitiger Autorisierung.

Zusätzliche Umsetzungsergebnisse in M2 Neben der reinen Migration wurden auch Entwicklungsstandards nachgezogen: TypeScript-Typen für zentrale Datenobjekte, modularisierte Styling-Struktur und ein reproduzierbarer Build-/Lint-Workflow. Diese Maßnahmen sind für den späteren Full-Stack-Ausbau relevant, weil sie Änderungen an API- und Datenmodellen kontrollierbar machen.

4.3 Meilenstein 3

Erweiterungen und Verbesserungen M3 erweitert die Anwendung von einer reinen Frontend-SPA zu einem vollständigen Websystem mit getrenntem Frontend/Backend, REST-API und persistenter Datenbank. Die Repository-Struktur wurde dafür in `frontend/` und `backend/` aufgeteilt.

Wesentliche Erweiterungen:

- **Eigenes Backend** mit Node.js/Express,
- **Persistenz** über SQLite + Prisma,

- **JWT-Authentifizierung** im HttpOnly-Cookie,
- **geschützte API-Endpunkte** mit Middleware,
- **Register/Login/Logout/User-Flow** zwischen Frontend und Backend,
- **Tests** für API- und Frontend-Kernlogik.

Architekturentscheidung: Trennung von UI und API Durch die Trennung in Frontend- und Backend-Schicht wurde die Verantwortung klar verteilt: das Frontend steuert Darstellung und Interaktion, das Backend validiert Eingaben, erzwingt Zugriffsregeln und kapselt Persistenzzugriffe. Diese Entscheidung verbessert Nachvollziehbarkeit, Sicherheit und Erweiterbarkeit.

Sicherheits- und Datenaspekte Im Gegensatz zu den vorherigen Ständen wurden Authentifizierungs- und Berechtigungsregeln explizit serverseitig umgesetzt:

- Token-Erzeugung und -Verifikation mit `jsonwebtoken`,
- Cookie-basierter Transport mit `httpOnly`,
- Passwort-Hashing mit `bcrypt`,
- geschützte Routen über Middleware (`authenticateToken`).

Zusätzlich wurde mit Prisma ein konsistentes, versionierbares Datenmodell eingeführt, das User, Campaigns, Memberships und Entity-Inhalte strukturiert abbildet.

Verwendete Vorlesungskonzepte M3 integriert Konzepte aus mehreren Vorlesungsblöcken:

- **REST-Design und HTTP-Methodik** (GET/POST/PUT/PATCH/DELETE),
- **Middleware-Architektur** in Express,
- **Authentifizierung/Autorisierung** mit JWT und Rollenmodell,
- **Persistenzmodellierung** mit relationalen Entitäten und ORM,
- **Automatisierte Tests** für Kernpfade.

Konkrete Codebereiche (Stand Meilenstein3)

- Backend-API und Auth-Middleware: `backend/src/server.ts`
- Datenmodell: `backend/prisma/schema.prisma`
- Frontend-Auth-Flow: `frontend/src/context/JWTAuthContext.tsx`
- Frontend-Routing inkl. Auth-Seiten: `frontend/src/App.tsx`

- Testbasis: `backend/src/server.test.ts`, `frontend/src/__tests__/auth.test.tsx`

Lessons Learned aus M3 Die Einführung eines eigenen Backends hat die Qualität der Gesamtarchitektur deutlich erhöht: kritische Regeln liegen nicht mehr im Client, Daten sind persistent, und Systemverhalten ist testbar abgesichert. Gleichzeitig wurde sichtbar, dass API-Verträge und Frontend-Typen eng aufeinander abgestimmt sein müssen, damit Änderungen ohne Seiteneffekte integriert werden können.

Technischer Reifegrad nach M3 Mit Abschluss von M3 war die Anwendung funktional bereits end-to-end nutzbar: Registrierung/Anmeldung, geschützte API-Routen, rollenabhängige Bearbeitung und persistente Speicherung waren durchgängig vorhanden. Damit verschob sich der Schwerpunkt von Feature-Implementierung auf Betriebssicherheit, Reproduzierbarkeit und Projektdokumentation — der inhaltliche Fokus von M4.

4.4 Meilenstein 4

Ergebnis von M4 Der vierte Meilenstein hat die bereits funktionsfähige Full-Stack-Anwendung in einen abgabefähigen Betriebszustand überführt. Im Mittelpunkt standen dabei nicht primär neue Kernfeatures, sondern **Deployment-Nähe, Reproduzierbarkeit, Demo-Fähigkeit und Abschlussdokumentation**.

Abgeschlossene Arbeitspakete Die zentralen M4-Arbeitspakete wurden in vier Blöcken abgeschlossen:

- **Betrieb vereinheitlicht:** reproduzierbarer Start der Gesamtanwendung über `docker compose up` mit Frontend-, Backend- und Datenbankpfad-Setup.
- **README vervollständigt:** Projektbeschreibung, Architektur, Setup-Schritte, Testhinweise, bekannte Einschränkungen.
- **Test- und Qualitätsnachweis konsolidiert:** strukturierte Darstellung der Teststrategie und aktueller Ergebnisstände.
- **Ausarbeitung finalisiert:** Kapitel 1–7 sowie Anhang mit formalem Abgabeformat.

VL-Bezug (Deployment/Betrieb) Der explizite Vorlesungsbezug von M4 liegt im Bereich Deployment/Betrieb: entscheidend ist die Fähigkeit, dass eine externe Person die Anwendung reproduzierbar in kurzer Zeit starten und bewerten kann. Dies wurde im Projekt über zwei Wege abgesichert:

1. containerisierter Start über `Compose`,
2. zusätzlich dokumentierter lokaler Fallback-Betrieb ohne Docker.

Nachvollziehbare Abgabefähigkeit Für die Abschlussabgabe wurden die geforderten Artefakte systematisch zusammengeführt: Repository-Finalstand, vollständiges README, Demo-Basis, PDF-Ausarbeitung und betriebsrelevante Hinweise. Damit ist das Projekt nicht nur entwicklungsseitig umgesetzt, sondern auch als bewertbares Produktpaket konsistent vorbereitet.

Lessons Learned aus M4 Ein technisch funktionsfähiges System ist nicht automatisch abgabefähig. Für den Projekterfolg waren in der Abschlussphase besonders wichtig: klare Betriebsanleitungen, robuste Startprozesse, konsistente Dokumentation und die enge Kopplung von Implementierung und Nachweis (Tests, Kapitel, Screenshots, Demo-Ablauf).

4.5 Zuordnung von VL-Konzepten zum Code

Die folgende Tabelle ordnet zentrale Vorlesungskonzepte den realisierten Artefakten zu. Die Zuordnung basiert auf den getaggten Meilenstein-Ständen und den daraus hervorgegangenen Modulen einschließlich des finalen Betriebsstands.

Zusammenfassung der Umsetzungslogik Die Meilensteinfoolge folgt einer klaren technischen Lern- und Reifungslogik: M1 etablierte das UI-Fundament, M2 industrialisierte die Frontend-Struktur mit React/TypeScript, M3 vervollständigte das System durch Backend, Authentifizierung, Datenhaltung und Tests, und M4 führte diese Ergebnisse in einen reproduzierbaren, abgabefähigen Betriebsstand über. Damit wurden die Vorlesungskonzepte nicht isoliert, sondern als zusammenhängende Systemarchitektur umgesetzt.

VL-Konzept	Meilenstein	Codebereich	Umsetzung im Projekt
Semantisches HTML	M1	src/about.html, src/login.html	Strukturierung über main, section, nav, formulargerechte Labels/Inputs.
Responsive Design	M1	src/styles/navigation.css	Mobile Sidebar per Media Query (max-width: 800px) und Overlay-Logik.
DOM-Interaktion mit JS	M1	src/script.js	Zustandswechsel der Navigation über Klassen, ARIA-Attribute und inert.
Build-Tooling	M2	package.json, vite.config.js	Dev-Server/Build-Pipeline mit Vite als Grundlage für SPA-Entwicklung.
TypeScript-Typisierung	M2	src/types/*, src/context/AuthContext.tsx	Typisierte Daten- und Context-Verträge zur Reduktion von Integrationsfehlern.
Komponentenarchitektur	M2	src/components/*	Zerlegung in wiederverwendbare UI-Bausteine statt seitenweiser Duplikation.
React Router	M2/M3	src/App.tsx, später frontend/src/App.tsx	Clientseitige Navigation mit dynamischen Routen für Campaign- und Entity-Ansichten.
State-Management (Hooks)	M2	src/pages/CampaignOverview.tsx	useState/useEffect für Suche, Sichtbarkeit und UI-Interaktionen.
Geteilter Zustand (Context)	M2/M3	AuthContext bzw. JWTAuthContext	Zentrale Verwaltung von Rollen/Session-Informationen über den Komponentenbaum.
REST-API-Design	M3	backend/src/server.ts	Ressourcenorientierte Endpunkte und methodenkonforme CRUD-/Auth-Operationen.
Middleware-Prinzip	M3	authenticateToken in backend/src/server.ts	Vorverarbeitung geschützter Requests inkl. JWT-Prüfung.
Authentifizierung mit JWT	M3	backend/src/server.ts, frontend/src/context/JWTAuthContext.tsx	Login erzeugt Token; geschützte Requests nutzen Cookie-basierten Auth-Flow.
Passwortsicherheit	M3	backend/src/server.ts	Serverseitiges Hashing und Vergleich über bcrypt.
Persistenz/ORM	M3	backend/prisma/schema.prisma	Relationales Modell für User, Campaigns, Rollen und Inhalte via Prisma + SQLite.
Automatisierte Tests	M3/M4	backend/src/server.test.ts, frontend/src/__tests__/auth.test.tsx	Absicherung zentraler Backend-/Frontend-Pfade mit Vitest, Supertest, Testing Library.
Deployment/Betrieb	M4	docker-compose.yml, README.md, dokumentation/sections/06_betrieb.tex	Reproduzierbarer Startprozess, dokumentierte Betriebsanleitung und Abgabevorbereitung.

Tabelle 2: Zuordnung zentraler Vorlesungskonzepte zu Codebereichen

5 Testing & Qualitätssicherung

5.1 Teststrategie

Die Qualitätssicherung kombiniert drei Ebenen: **Unit-nahe Logiktests**, **API-Integrationstests** und **Frontend-Komponenten-/Flow-Tests**. Ziel ist nicht nur die Prüfung einzelner Funktionen, sondern vor allem die Absicherung der kritischen Nutzerpfade: Registrierung, Login, Rollen-/Rechteprüfung und Campaign-Operationen.

Backend-Strategie Im Backend werden Routen mit `supertest` gegen die Express-App getestet. Dadurch lassen sich HTTP-Statuscodes, Response-Bodies, Cookie-Handling und Autorisierungsregeln realitätsnah prüfen. Die Prisma-Zugriffe werden in den Tests gezielt gemockt, sodass Fehlerfälle (z. B. Unique-Constraint-Verletzungen) reproduzierbar testbar sind.

Frontend-Strategie Im Frontend werden mit Vitest + Testing Library Seiten und Context-Logik getestet. Fokus liegt auf sichtbar nutzerrelevantem Verhalten: Formularvalidierung, API-Aufrufe, Navigation sowie Fehler- und Erfolgsfälle. Externe Abhängigkeiten (`fetch`, Routing) werden kontrolliert gemockt.

Warum diese Aufteilung Die Kombination aus API-Integrationstests und Frontend-Interaktionstests liefert ein gutes Verhältnis aus Aussagekraft und Wartbarkeit: Routenfehler, fehlende Berechtigungsprüfungen und fehlerhafte Client-Flows werden früh erkannt, ohne den Aufwand eines vollständigen End-to-End-Teststacks.

5.2 Was wird getestet

Backend-Abdeckung Die Backend-Tests decken zentrale Sicherheits- und Geschäftsregeln ab:

- **Authentifizierung:** Login erfolgreich/fehlgeschlagen, JWT-Cookie wird gesetzt, Logout liefert erwartete Antwort.
- **Autorisierung:** geschützte Endpunkte liefern `401` ohne Token und `403` bei ungültigem Token.
- **Rollenrechte in Campaigns:** Editor darf Join-Code nicht regenerieren und keine Member-Rollen ändern; DM darf beide Operationen ausführen.
- **Registrierung:** Passwortregeln und Duplikatbehandlung (`409` bei bereits registrierter E-Mail).
- **Join-Code-Logik:** Format des Join-Codes sowie Retry-Verhalten bei Unique-Konflikten.

Frontend-Abdeckung Die Frontend-Tests prüfen sowohl Seitenlogik als auch API-Helfer:

- **Auth-Context:** `login()` und `logout()` rufen die richtigen API-Endpunkte mit korrekten Request-Optionen auf.
- **Register-Flow:** schwache Passwörter werden clientseitig blockiert, starke Passwörter führen zum Register-Request.
- **Home-Flow:** Campaigns laden, Join per Code und Weiterleitung auf die Overview-Seite.
- **API-Wrapper (`apiFetch`):** `credentials: include`, Header-Handling und Fehler-Mapping (JSON-error bzw. Fallback-Text).

5.3 Exemplarische Ergebnisse

Die folgenden Auszüge stammen aus den aktuellen Testläufen:

```
cd backend
npm test

Test Files  4 passed (4)
Tests       21 passed (21)
```

Listing 1: Backend-Testlauf

```
cd frontend
npm test

Test Files  5 passed (5)
Tests       11 passed (11)
```

Listing 2: Frontend-Testlauf

```
# Backend (Autorisierungs- und Logout-Fälle)
npm --prefix backend test -- src/server.routes.test.ts
# Ergebnis: 11 passed

# Frontend (API-Wrapper-Tests)
npm --prefix frontend test -- src/__tests__/api.test.ts
# Ergebnis: 3 passed
```

Listing 3: Ausgewählte gezielte Testläufe

Interpretation Die Ergebnisse bestätigen, dass die zentralen Sicherheits- und Geschäftsregeln stabil umgesetzt sind: Auth-Cookie-Flow funktioniert erwartungsgemäß, unautorisierte Zugriffe werden korrekt abgefangen und rollenbasierte Schreibrechte serverseitig durchgesetzt. Auf Frontend-Seite zeigen die Tests, dass API-Aufrufe konsistent aufgebaut werden und Fehler dem UI in auswertbarer Form zur Verfügung stehen.

6 Betrieb

6.1 Start der Anwendung

Für den Entwicklungsbetrieb ist **Docker Compose** der primäre Startweg, da Frontend, Backend und Persistenzpfad in einem reproduzierbaren Setup zusammengeführt werden.

Betrieb mit Docker Compose Der Start erfolgt mit den folgenden Befehlen:

```
cp .env.example .env
docker compose up
```

Nach dem Start stehen die zentralen Endpunkte bereit:

- Frontend: <http://localhost:5173>
- Backend: <http://localhost:3000>
- Health-Endpoint: <http://localhost:3000/api/health>

Im Compose-Betrieb führt der Backend-Service vor dem Start automatisch `prisma migrate deploy` aus. Damit ist sichergestellt, dass das Datenbankschema dem aktuellen Migrationsstand entspricht.

Lokaler Betrieb ohne Docker Alternativ kann die Anwendung lokal in zwei Prozessen betrieben werden:

```
cd backend
npm ci
npx prisma migrate deploy
npm run dev
```

Listing 4: Backend lokal starten

```
cd frontend
npm ci
npm run dev
```

Listing 5: Frontend lokal starten (zweites Terminal)

Im lokalen Betrieb ist ein `backend/.env` mit den notwendigen Variablen erforderlich (siehe section 6.2).

6.2 Konfiguration

Die Anwendung wird über Umgebungsvariablen konfiguriert. Für den Compose-Betrieb dient `.env.example` als Vorlage.

Variable	Beispielwert	Bedeutung
JWT_SECRET	dev-secret-change-me	Secret für Signierung und Verifikation der JWTs im Backend.
CORS_ORIGIN	http://localhost:5173	Erlaubte Origin(s) für CORS-Requests auf die API.
DATABASE_URL	file:./data/dev.db	SQLite-Datenbankpfad des Backends.
VITE_API_PROXY_TARGET	http://backend:3000	Ziel für den /api-Proxy des Vite-Dev-Servers.

Tabelle 3: Relevante Umgebungsvariablen im Entwicklungsbetrieb

Voraussetzungen Für den Betrieb werden Node.js und npm benötigt. Im Docker-Modus sind zusätzlich Docker Engine und Docker Compose erforderlich. Die Persistenz erfolgt auf SQLite-Basis und benötigt keinen separaten Datenbankserver.

6.3 Betriebsmodus

Die Anwendung ist als **Entwicklungs- und Demo-Betrieb** ausgelegt. Frontend und Backend laufen als getrennte Prozesse/Container, kommunizieren über HTTP und nutzen ein Cookie-basiertes Session-Modell mit JWT.

Typischer Workflow

1. Services starten (Compose oder lokal).
2. Benutzer über /register anlegen und anmelden.
3. Campaign erstellen oder per Join-Code beitreten.
4. Inhalte pflegen (abhängig von der Rolle DM/EDITOR/PLAYER).

Betriebsrelevante Eigenschaften

- Daten werden persistent in SQLite gespeichert.
- Schemaänderungen werden über Prisma-Migrationen ausgerollt.
- Geschützte Endpunkte erfordern gültige JWT-Cookies.
- Die Trennung von Frontend und Backend erlaubt klare Fehlersuche (UI-/API-/DB-Ebene).

7 Reflexion & Fazit

7.1 Was lief gut

Ein wesentlicher Erfolgsfaktor war die schrittweise Entwicklung über klar abgrenzte Meilensteine. Dadurch konnten Anforderungen iterativ umgesetzt und laufend validiert werden: zuerst ein stabiles UI-Grundgerüst, danach die Migration auf eine komponentenbasierte SPA und schließlich die Erweiterung zur Full-Stack-Lösung mit Backend, Authentifizierung und Datenbank. Diese Reihenfolge war rückblickend sinnvoll, weil sie technische Risiken reduzierte und zu jedem Zeitpunkt einen lauffähigen Projektstand sicherstellte.

Auch die Verbindung von fachlicher Domäne und technischer Umsetzung hat gut funktioniert. Da das Projekt aus einem realen Bedarf im eigenen Spielkontext entstanden ist, waren Anforderungen wie Übersichtlichkeit, schnelle Bedienbarkeit und rollenbasierte Sichtbarkeit von Beginn an klar. Das hat Priorisierungsentscheidungen erleichtert und zu einer konsistenten Produktlinie geführt.

In der Teamarbeit war positiv, dass technische Umbauten nicht nur feature-getrieben, sondern auch strukturell durchgeführt wurden. Beispiele dafür sind die Umstellung auf wiederverwendbare Komponenten, die Einführung von TypeScript-Typen, die spätere Trennung in Frontend/Backend und die klare Dokumentation der Meilensteine. So wurde vermieden, dass kurzfristige Lösungen die Wartbarkeit dauerhaft verschlechtern.

Zusätzlich war die frühe Etablierung von Tooling hilfreich: Linting/Formatierung, automatisierte Tests und ein reproduzierbarer Startprozess mit Docker Compose haben die Qualität in der Abschlussphase messbar stabilisiert. Gerade bei der Vorbereitung von Demo und Ausarbeitung hat sich gezeigt, dass ein konsistenter Betriebsstand viele Folgekosten reduziert.

7.2 Was würden wir anders machen

Rückblickend würde das Projekt einige technische Entscheidungen früher treffen. Die wichtigste davon betrifft die Systemgrenze zwischen Frontend und Backend. In den frühen Ständen wurden Teile der Logik zunächst clientseitig modelliert; mit wachsender Komplexität (Rollen, Authentifizierung, persistente Inhalte) mussten diese Konzepte später auf serverseitige Regeln umgestellt werden. Beim nächsten Projekt würde die API- und Datenmodellplanung früher erfolgen, um Umbauten in späteren Phasen zu reduzieren.

Ein zweiter Punkt betrifft die Benennung und Konsistenz von Pfaden, Modulen und Routen. Durch den langen Entwicklungsverlauf mit mehreren Umstrukturierungen (`src/` → `frontend/`, neue Backend-Struktur, Refactorings im Routing) entstand zwischenzeitlich inkonsistente Terminologie. Diese wurde zwar bereinigt, hätte aber mit früheren Konventionen weniger Nacharbeit erzeugt.

Auch die Teststrategie würde künftig früher breiter aufgesetzt. Die vorhandenen Tests decken zentrale Pfade gut ab, wurden aber erst in späteren Meilensteinen systematisch ausgebaut.

Ein früheres, stärker risikobasiertes Testdesign (z. B. Auth- und Rollenfälle bereits parallel zur ersten Backend-Integration) hätte manche Fehlerkorrekturen beschleunigt.

Organisatorisch würde die Abschlussphase zeitlich noch stärker gegen Ende gepuffert werden. Der inhaltliche Abschluss (Demo, Kapitelabgleich, formale Nacharbeit) ist erfahrungsgemäß aufwendiger als erwartet, selbst wenn die Kernfunktionalität bereits steht. Für zukünftige Projekte ist deshalb eine frühere Synchronisation zwischen Implementierungsstand und finalem Abgabeformat sinnvoll.

7.3 Was wurde gelernt

Technisch wurde vor allem deutlich, wie wichtig eine saubere Verantwortungstrennung in Webanwendungen ist. Die Trennung zwischen UI, API und Persistenz verbessert nicht nur die Wartbarkeit, sondern auch Sicherheit und Nachvollziehbarkeit. Insbesondere bei Authentifizierung, Rollenlogik und Datenzugriff ist eine serverseitige Durchsetzung zentral, da rein clientseitige Regeln nicht ausreichen.

Ein weiterer Lernpunkt ist der praktische Wert konsistenter Typisierung. TypeScript hat die Kommunikation zwischen Modulen verbessert und Refactorings abgesichert, vor allem bei komplexeren Datenstrukturen für Kampagneninhalte und Benutzerzustände. In Kombination mit klaren Interfaces und einem strukturierten API-Vertrag konnten Integrationsprobleme früh erkannt werden.

Im Bereich Qualitätssicherung wurde gelernt, dass Tests den größten Nutzen haben, wenn sie reale Kernabläufe abbilden: Login/Logout, geschützte Routen, Rollenrechte und typische Nutzeraktionen. Diese Tests sind nicht nur technische Absicherung, sondern auch eine ausführbare Spezifikation des erwarteten Systemverhaltens.

Organisatorisch hat das Projekt gezeigt, dass kontinuierliche Dokumentation ein entscheidender Erfolgsfaktor ist. Wenn Architektur-, Betriebs- und Testentscheidungen erst am Ende beschrieben werden, geht Kontext verloren. Die laufende Pflege von README, Meilensteinzuordnung und Ausarbeitung erleichtert dagegen sowohl Teamabstimmung als auch Abgabevorbereitung erheblich.

Insgesamt wurde aus dem Projekt mitgenommen, dass ein gutes Ergebnis weniger von einzelnen Features abhängt als von der Verbindung aus **inkrementeller Planung, strukturiertem Refactoring, testbarer Architektur und reproduzierbarem Betrieb**. Diese Kombination hat aus einem anfänglich kleinen Frontend-Prototyp eine vollständig nutzbare Full-Stack-Anwendung gemacht.

A Anhang: Screenshots der fertigen App

Zielumfang: ca. 2–3 Seiten.

Hier werden Screenshots der finalen Anwendung mit kurzen Bildunterschriften eingebunden.

Abbildung 5: Platzhalter: Übersichtsseite der Anwendung

B Formales und Quellenhinweise

B.1 Formale Anforderungen

- Ausgabe als PDF mit sauberem Layout.
- Deckblatt mit Namen, Matrikelnummern, Projekttitel und Abgabedatum.
- Quellenangaben bei maßgeblich genutzten externen Ressourcen.

B.2 Quellen

Alle externen Quellen werden im Literaturverzeichnis und bei Bedarf direkt im Fließtext referenziert.